객체지향 프로그래밍 고급

6강 : 복사생성자

[예제]

다음은 SoSimple 클래스이다. 정수 2를 더하는 Adder()함수 호출 문장을 완성하시오.

```
class SoSimple
{
private:
    int num;
public:
    SoSimple(int n) : num(n)
    {
    }
    SoSimple& Adder(int n)
    {
        num += n;
        return *this;
    }
    void ShowData()
    {
        cout << num << endl;
    }
};
```

```
int main()
{
    SoSimple obj(3);
    [                                        ]

    obj.ShowData();
    ref.ShowData();

    return 0;
}
```

Chapter 5

복사생성자

깊은 복사와 얕은 복사

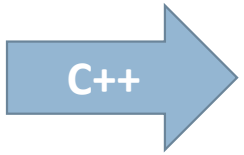
복사생성자의 호출

static

C++ 스타일의 초기화

변수와 참조자의 선언 및 초기화 방식

```
int num = 20;
int &ref = num;
```



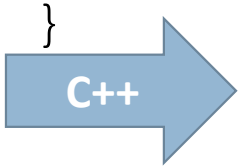

```
int num(20);
int &ref(num);
```

C++ 스타일의 초기화 - 클래스

객체의 선언 및 초기화

```
class SoSimple
{
private:
	int num1;
	int num2;
public:
	SoSimple(int n1, int n2) : num1(n1), num2(n2)
	{ }
	void showData()
	{
		cout << num1 << endl;
		cout << num2 << endl;
	}
};
```

```
int main()
{
	SoSimple sim1(15, 20);
	SoSimple sim2 = sim1;
	sim2.showData();
	return 0;
}
```



SoSimple sim2(sim1);

C++ 스타일의 초기화 – 클래스

객체의 선언 및 복사

SoSimple sim2(sim1); 의 의미

- SoSimple형 객체 sim2 를 생성한다. .
- 이 때 sim2는 객체 sim1을 인자로 받는 생성자를 호출한다.

복사 생성자

객체 선언 및 복사(클래스형 객체2(객체1);)

- 같은 클래스형을 갖는 객체와 다른 객체 간에 대입연산자(=)를 사용하여 모든 멤버 변수의 데이터를 비트 단위로 복사한다.

복사 생성자

- 복사 생성자는 동일한 클래스형을 갖는 객체는 하나의 매개변수로 전달받아 값을 복사함으로써 다른 객체를 초기화하는 생성자이다.
- 멤버 대 멤버의 복사를 한다.
- 일반 대입 연산자에 사용할 수 없고 초기화할 때만 사용한다.
- 복사 생성자를 정의하지 않으면 디폴트 복사 생성자가 자동으로 삽입된다.

복사 생성자의 형식

```
클래스명(const 클래스명& 객체명)
```

- 복사 생성자는 매개변수가 하나이며, 자기 클래스에 대한 참조(&)를 인자로 받는다.
- 인자값을 변경시킬 일이 없으면 const를 붙여 상수 처리하고, 변경시킬 경우 const를 사용하지 않는다.
- &객체명은 참조자로 객체 원본을 사용한다는 의미이다.

복사 생성자의 예

```
class SoSimple
{
private:
    int num1;
    int num2;
public:
    SoSimple(int n1, int n2) {
        num1 = n1;
        num2 = n2;
    }
    void ShowData() {
        cout << "num1:" <<num1
    <<'₩t'<<"num2"  <<num2<<endl;
    }
```

```
    SoSimple(const SoSimple &copy)
    {
        num1 = copy.num1;
        num2 = copy.num2;
    }
```

복사 생성자의 예

```
class SoSimple
{
private:
	int num1;
	int num2;
public:
	SoSimple(int n1, int n2) {
		num1 = n1; num2 = n2;
		cout << "1.Called SoSimple(n1, n2)₩n";
	}
	SoSimple(SoSimple &copy)
		: num1(copy.num1), num2(copy.num2) {
		cout << "3.Called SoSimple(&copy)₩n";
	}
	void ShowData() {
		cout << "num1:" <<num1 <<'₩t'<<"num2"
	<<num2<<endl;
	}
```

```
int main()
{
	SoSimple sim1(15, 20);
	sim1.ShowData();
	cout << "2." << endl;
	SoSimple sim2 = sim1;
	sim2.ShowData();
	return 0;

}
```

기본 복사 생성자

“복사 생성자를 정의하지 않으면, 멤버 대 멤버의 복사를 진행하는 기본 복사 생성자가 자동으로 삽입된다.

Chapter 5

복사생성자

깊은 복사와 얕은 복사

복사생성자의 호출

static

얕은 복사

- 멤버 대 멤버의 복사를 하는 디폴트 복사 생성자를 얕은 복사(shallow copy)라 한다. (디폴트 복사 생성자)

얕은 복사의 문제점

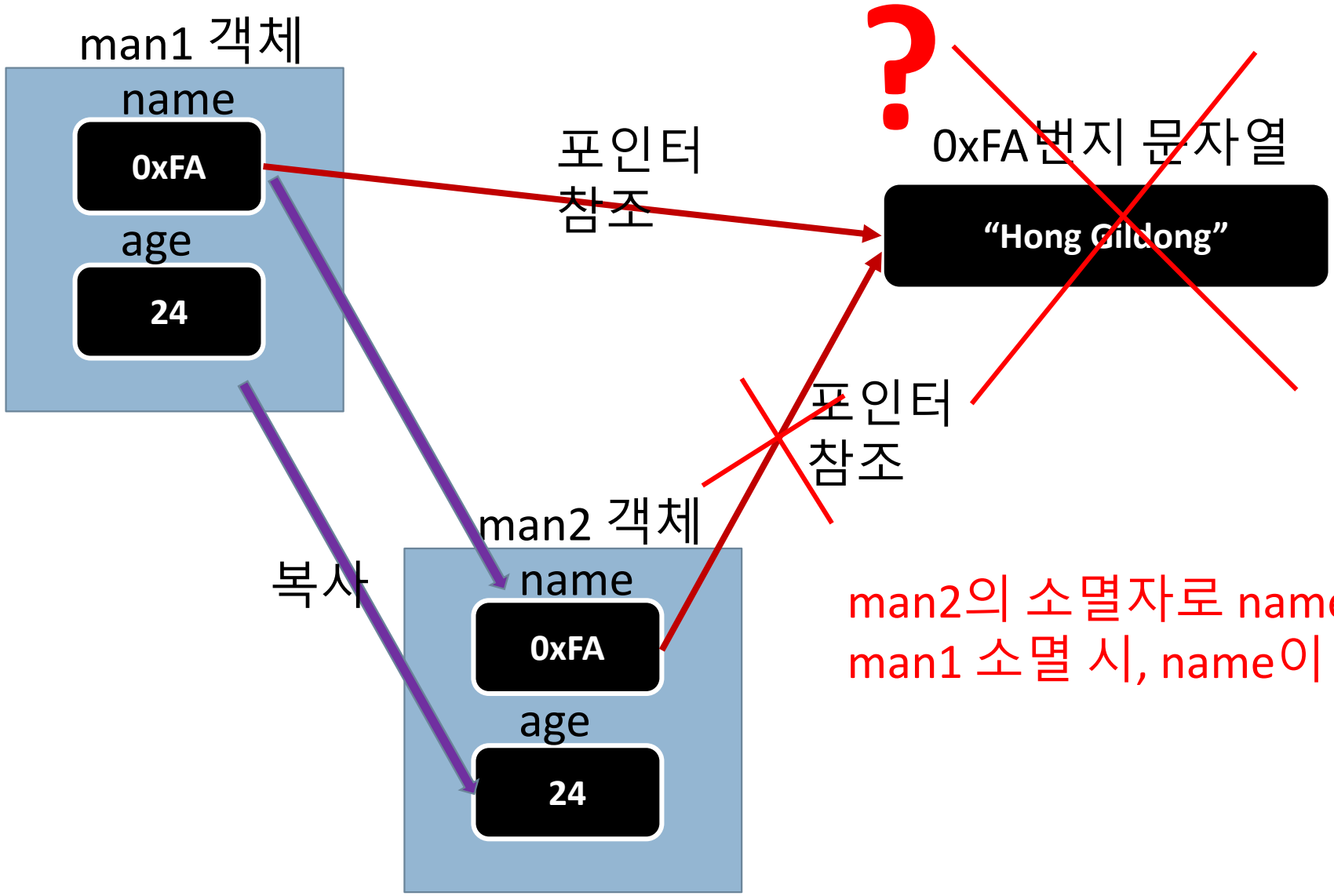
```
class Person
{
private:
	char *name;
	int age;
public:
	Person(char *myname, int myage)
	{
		int len = strlen(myname) + 1;
		name = new char[len];
		strcpy(name, myname);
		age = myage;
	}
	void ShowData() const {
		cout << name << '\t' << age << endl;
	}
	~Person()
	{
		delete[]name;
		cout << "called  destruction\n";
	}
};

int main()
{
	Person man1("Hong Gildong", 24);
	Person man2 = man1;
	man1.ShowData();
	man2.ShowData();
	return 0;
}
```

얕은 복사의 문제점

- 앞의 예제는 왜 오류가 발생하는가?
  - 개발자가 복사 생성자를 정의하지 않았기 때문에 컴파일러가 기본 복사 생성자를 자동으로 삽입하여 멤버 변수를 초기화하였다.
  - 기본 복사 생성자는 직접 값을 복사하는 것이 아니라 참조만 한다.
  - 즉, 객체가 실제로 데이터를 가지고 있는 것이 아니다.
  - 두 개의 객체(man1, man2)가 하나의 멤버 변수(myname)를 공유하고 있어 두 개의 소멸자가 각각 실행될 때 해제할 메모리가 존재하지 않아 메모리 누수 현상이 생긴다.

얕은 복사의 문제점



man2의 소멸자로 name이 delete됨
man1 소멸 시, name이 없어 오류 발생

깊은 복사

- 앞의 예제에서 man2의 소멸자가 실행되면서 멤버 변수에 할당된 동적 메모리를 해제할 때 man1의 동적 메모리도 해제된다.
- man1의 소멸자가 실행될 때 해제할 메모리가 없으므로 메모리 누수 현상이 발생한다.
- 디폴트 복사 생성자는 얕은 복사만 하므로 멤버 변수가 동적 할당을 하는 new 연산자를 사용하는 경우에는 개발자가 직접 복사 생성자를 정의하여 메모리를 새로 할당해서 내용을 복사해야 한다.
- 이것을 깊은 복사(deep copy) 라고 한다.

깊은 복사

```
Person(const Person &copy)
{
    int len = strlen(copy.name) + 1;
    name = new char[len];
    strcpy(name, copy.name);
    Age = myage;
}
```

[예제] 복사 생성자와 소멸자를 추가하라.

```
class Student
{
private:
	char *ID;
	char *name;
	int Kor;
	int Math;
public:
	Student(char *ID, char *name, int Kor, int Math)
	{
		this->ID = new char[strlen(ID) + 1];
		strcpy(this->ID, ID);
		this->name = new char[strlen(name)+1];
		strcpy(this->name, name);
		this->Kor = Kor;
		this->Math = Math;
	}
	void ShowData() { }
	~Student() { }
```

```
int main()
{
	char mid[10], mname[20];
	int mkor, mmath;

	cin >> mid >> mname >> mkor >> mmath;
	Student std1(mid, mname, mkor, mmath);
	Student std2 = std1;
	std1.ShowData();
	std2.ShowData();
	return 0;
}
```

Chapter 5

복사생성자

깊은 복사와 얕은 복사

복사생성자의 호출

static

깊은 복사

- 복사 생성자는 클래스에 멤버 변수가 포인터 변수를 포함하고 있으면 반드시 복사 생성자를 정의해 주어야 한다.

복사생성자는 언제 호출되는가?

Case1: 기존에 생성된 객체를 이용하여 새로운 객체를 초기화하는 경우

Case2: Call-by-Value 방식의 함수호출과정에서 객체를 인자로 전달하는 경우

(단, 참조에 의한 호출인 경우, 복사 생성자가 호출되지 않고 메모리 번지만 전달이 된다.

Case3: 함수가 객체를 반환할 때(참조형으로 반환하지 않는 경우)

공통점

메모리의 공간 할당과 초기화가 동시에 일어나는 경우 :

객체를 새로 생성해야 한다.

단, 생성과 동시에 동일한 자료형의 객체로 초기화해야 한다."

복사 생성자가 자동 호출되는 경우

Case 1:

```
Person man1("Hong Gildong", 24);
Person man2 = man1;
```

- man2의 메모리 공간 할당과 동시에 man1에 저장된 값으로 초기화

복사 생성자의 호출(case 2)

```
SoSimple SimpleFunc(SoSimple ss)
{   . . .   }

int main() {
    SoSimple obj;
    SimpleFunc(obj);
    . . .
}
```

- SimpleFunc()이 호출되는 순간 매개변수 ss는 메모리 할당과 동시에 변수 obj에 저장되어 있는 값으로 초기화

복사 생성자의 호출(case 2)

```
class SoSimple
{
private:
	int num;
public:
	SoSimple(int n) : num(n) {}
	SoSimple(const SoSimple& copy) :
				num(copy.num)
	{
		cout << "Called SoSimple& copy)\n";
	}
	void ShowData()
	{
		cout << "num: " << num << endl;
	}
};
```

```
void SimpleFunc(SoSimple ss)
{
	ss.ShowData();
}
int main()
{
	SoSimple obj(7);
	cout << "함수호출전\n";
	SimpleFunc(obj);
	cout << "함수호출후\n";
	return 0;
}
```

복사 생성자의 호출(case 3)

```
SoSimple SimpleFunc(SoSimple ss)
{
   . . .
   return ss;
}
```

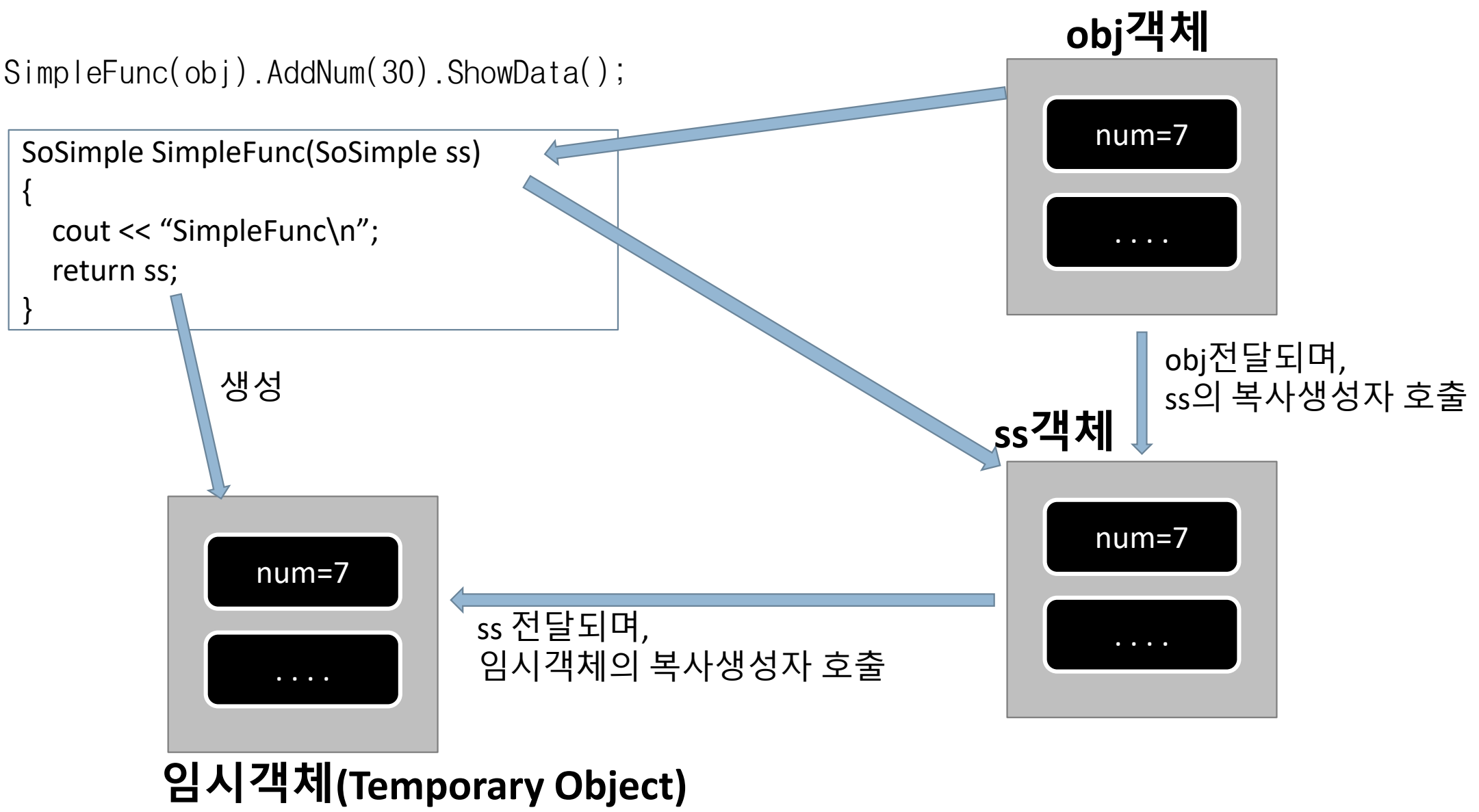
- 함수가 값을 반환하면, 별도의 메모리 공간이 할당되고, 이 공간에 반환값이 저장된다.

복사 생성자의 호출(case 3)

```
class SoSimple
{
private:
	int num;
public:
	SoSimple(int n) : num(n)
	{}
	SoSimple(const SoSimple& copy) : num(copy.num)
	{
		cout << "Called SoSimple(const SoSimple&
		copy)\n";
	}
	SoSimple& AddNum(int n)
	{
		num += n;
		return *this;
	}
	void ShowData()
	{
		cout << "num: " << num << endl;
	}
};
SoSimple SimpleFunc(SoSimple ss)
{
	cout << "SimpleFunc\n";
	return ss;
}
int main()
{
	SoSimple obj1(7);
	obj1.ShowData();
	SoSimple obj2 = SimpleFunc(obj1);
	obj2.ShowData();
	obj2.AddNum(30).ShowData();
	return 0;
}
```

```
SimpleFunc(obj).AddNum(30).ShowData();
```

```
SoSimple SimpleFunc(SoSimple ss)
{
    cout << "SimpleFunc\n";
    return ss;
}
```



[예제]복사 생성자를 사용해 다른 객체를 초기화하기

이름, 나이, 몸무게를 갖는 Dog 클래스를 정의하고 멤버변수 name을 동적할당하시오. 아래 main()함수가 성립되도록 복사생성자를 추가하시오.

```
class Dog
{
private:
    char * name;
    int age;
    int weight;
public:
    Dog(char * name, int age, int weight);
    void ShowData();
     . . .
};
```

```
int main()
{
    Dog obj1(“happy”, 3, 4);
    Dog obj2 = obj1;
    obj1.ShowData();
    obj2.ShowData();
    return 0;
}
```

[예제]

DynamicArray 클래스는 integer형 배열을 만들기 위한 pointer를 멤버변수로 갖는다. 멤버함수 SetData(int index, int value)는 arr 배열의 index번째에 value값을 할당하는 함수이다. main()이 실행되도록 클래스를 완성하라.

```
class DynamicArray
{
private:
	int* arr;
	int size;
public:
	DynamicArray(int arrSize);
	~DynamicArray();
	void ShowData();
	bool SetData(int index, int value);
};
```

```
int main()
{
    int size;
    cout << "0보다 큰 정수를
    입력하시오." << endl;
    cin >> size;
    DynamicArray da1(size);
    for (int i = 0; i < size; i++) {
        if (!da1.SetData(i, i + 1))
        {
            cout << "index 오류\n";
            return 0;
        }
    }
    da1.ShowData();
    DynamicArray da2 = da1;
    for (int i = 0; i < size; i++) {
        if (!da2.SetData(i, i - 1))
        {
            cout << "index 오류\n";
            return 0;
        }
    }
    da2.ShowData();
    return 0;
}
```

[예제] 다음 프로그램을 완성하라.

책값이 만원미만이면 책을 사고, 이상이면 책을 사지 않겠다고 출력하는 프로그램을 구현하라.

```
class Book
{
private:
	char* title;
	int price;
public:
	Book(const char * title, int price);
	~Book();
	void ShowData();
	bool SetData(char * title, int price);
	int GetPrice();
};
```

```
int main()
{
	Book b("C++", 11000);
	if (GetPrice(b) <10000 )
		cout << "책을 사지 않겠습니다. <<endl;
	else
		cout << "책을 사겠습니다. " << endl;
}

int GetPrice(Book obj)
{
	return obj.GetPrice();
}
```

[과제2] Banking System 2.0

Banking System 1.0 을 다음과 같이 업그레이드하라.

Account 클래스를 정의하시오.

- 고객이름은 문자열포인터로 선언하고 동적할당의 형태로 구현하라.
- 복사생성자를 추가하라.

```
int accID;
string regID;
char *name;
int balance;
```

Chapter 6

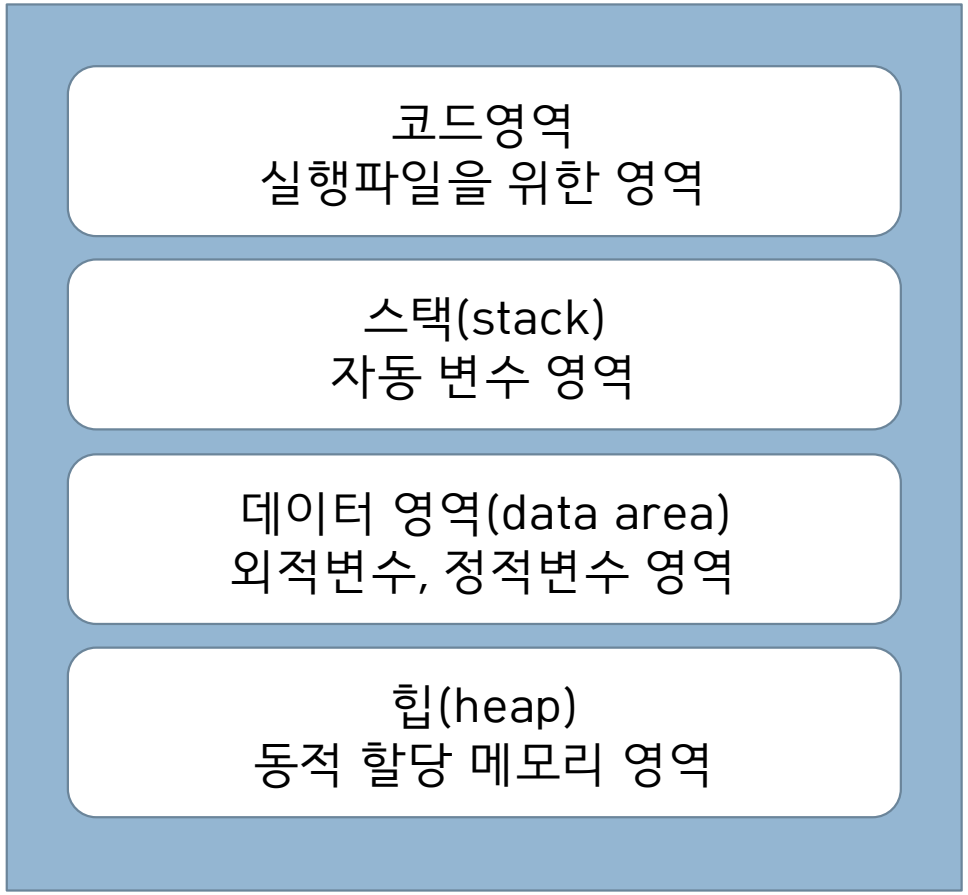
복사생성자

깊은 복사와 얕은 복사

복사생성자의 호출

static

메모리 영역 분류


코드영역
실행파일을 위한 영역
스택(stack)
자동 변수 영역
데이터 영역(data area)
외적변수, 정적변수 영역
힙(heap)
동적 할당 메모리 영역

static

- 정적멤버는 모든 객체가 공유하는 멤버이다.
- 멤버변수와 멤버함수들은 객체마다 하나씩 존재했으나, 정적멤버변수와 정적멤버함수는 클래스에 오직 하나만 존재한다.
- 전역변수처럼 main 함수가 호출되기 전에 메모리 공간에 적재되어 초기화된다.
- 선언된 클래스의 모든 객체 내에서 직접 접근이 가능하다.
- static 멤버는 반드시 초기화해야 한다.

static 멤버 변수

분류	멤버 변수	정적 멤버 변수
클래스		클래스마다 하나씩 복사(클래스소유)
객체	객체마다 하나씩 복사(객체소유)	모든 객체에서 공동 사용 가능

static 멤버 변수

- 정적 멤버 변수는 객체가 공유해서 사용하므로 생성자에서 초기화하지 않는다.

```
class Fn
{
private:
    static int count;
public:
    Fn() {
        count++;
        cout << count << endl;
    }
};
int Fn::count = 0;
```

```
int main()
{
    Fn f1;
    Fn f2;

    return 0;
}
```

static 멤버 함수

- static 멤버함수는 클래스에서 정의한 static 멤버 변수에 접근하는 함수
- 어느 객체에서도 호출할 수 있고 또는 직접 클래스명을 사용하여 호출할 수도 있다.
- private으로 선언된 static멤버변수는 해당 클래스 내부에서만 접근 가능하므로 이 멤버변수에 접근하는 방법은 static멤버함수를 사용하는 것이다.
- static 멤버함수는 일반멤버함수를 호출할 수 없다.

static 멤버 함수

```
class Fn
{
private:
    static int count;
public:
    Fn() {
        count++;
    }
    static void showFn();
};
int Fn::count = 0;
```

```
void Fn::showFn() {
    cout << "카운트: " << count << endl;
}

int main()
{
    Fn f1;
    Fn f2;
    Fn::showFn();
    return 0;
}
```

[예제]

학생정보를 가진 Student 클래스를 정의하고, 객체가 생성될 때마다 학생수가 증가하고 소멸할 때마다 학생수가 감소하는 정적멤버함수를 정의하시오.

학생정보(const)
  이름
  학번